Announcements

- Login, open an IDE, and Folio/browser for class
- Check grades in Folio
- Download binaryIO.zip and binaryData.zip

1

Announcements

- Revel Exercises due
  - Chap 17
- Project 8 due Friday, start of lab
  - See Program Submission requirements in Folio

2

Warm-Up Questions

- What other controls do we have access to?
- What other functionality do we have access to with these controls?
- How do we deal with events generated by these controls?
- How does JavaFX handle video and audio?
- What can we do with video and audio within a JavaFX program?

3

Lecture Questions

- What is the difference between binary and text files?
- What are the advantage/disadvantages to binary IO?
- How does Java accomplish binary input/output?

4

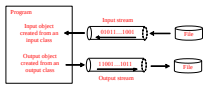
Chapter 17: Binary I/O

CSCI 1302 – Programming Principles II

5

Recall: Text I/O in Java

- File object encapsulates the properties of a file or a path, but does not contain the methods for reading/writing data from/to a file
- In order to perform I/O, need to create objects using appropriate Java I/O classes




```
Scanner input = new Scanner(new File("temp.txt"));
System.out.println(input.nextLine());
input.close();
PrintWriter output = new PrintWriter("temp.txt");
output.println("CSCI 1301");
output.close();
```

6

Text File vs. Binary File

- Data stored in a text file is represented in human-readable form
- Data stored in a **binary file** is represented in **binary form**
- Humans, generally, cannot read binary files
- Binary files are designed to be read by programs

7

Text File vs. Binary File

- Advantage of binary files is that they are more efficient to process than text files
- For example, Java source programs are stored in text files and can be read by a text editor, but Java classes are stored in binary files and are read by the JVM

8

Text File vs. Binary File

- Although it is not technically precise nor correct, imagine that a text file consists of a sequence of characters and a binary file consists of a sequence of bits
- For example, the decimal integer 199 might be stored as the sequence of three characters: '1', '9', '9' in a text file and the same integer might be stored as a byte-type value C7 in a binary file, because decimal 199 equals to hex C7

9

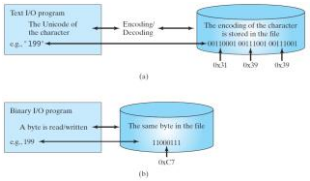
Binary I/O

- Text I/O requires encoding and decoding
- JVM converts Unicode to a file specific encoding when writing a character and converts a file specific encoding to Unicode when reading a character
- **IMPORTANT REMINDER:** Always close file streams and/or use try-with-resources to avoid data corruption or incomplete files

10

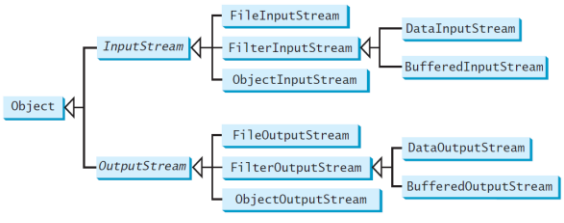
Binary I/O

- As mentioned, binary I/O does not require conversions
- When writing a byte to a file, original byte is copied into the file
- When reading a byte from a file, exact byte in the file is returned



11

Binary I/O Classes



12

InputStream

java.io.InputStream	The value returned is a byte as an int type
+read(): int	Reads the next byte of data from the input stream. The value byte is returned as an int value in the range 0 to 255. If no byte is available because the end of the stream has been reached, the value –1 is returned.
+read(b: byte[]): int	Reads up to b.length bytes into array b from the input stream and returns the actual number of bytes read. Returns -1 at the end of the stream.
+read(b: byte[], off: int, len: int): int	Reads bytes from the input stream and stores into b[off], b[off+1], …, b[off+len-1]. The actual number of bytes read is returned. Returns -1 at the end of the stream.
+available(): int	Returns the number of bytes that can be read from the input stream.
+close(): void	Closes this input stream and releases any system resources associated with the stream.
+skip(n: long): long	Skips over and discards n bytes of data from this input stream. The actual number of bytes skipped is returned.
+markSupported(): boolean	Tests if this input stream supports the mark and reset methods.
+mark(readlimit: int): void	Marks the current position in this input stream.
+reset(): void	Repositions this stream to the position at the time the mark method was last called on this input stream.

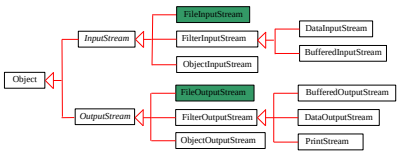
13

OutputStream

java.io.OutputStream	The value is a byte as an int type
+write(int b): void	Writes the specified byte to this output stream. The parameter b is an int value. (byte)b is written to the output stream.
+write(b: byte[]): void	Writes all the bytes in array b to the output stream.
+write(b: byte[], off: int, len: int): void	Writes b[off], b[off+1], …, b[off+len-1] into the output stream.
+close(): void	Closes this output stream and releases any system resources associated with the stream.
+flush(): void	Flushes this output stream and forces any buffered output bytes to be written out.

14

FileInputStream/FileOutputStream



- FileInputStream/FileOutputStream associates a binary input/output stream with an external file
- All the methods in FileInputStream/FileOutputStream are inherited from its superclasses

15

FileInputStream

- To construct a FileInputStream, use the following constructors:

```
public FileInputStream(String filename)
public FileInputStream(File file)
```

- A `java.io.FileNotFoundException` will occur if you attempt to create a FileInputStream with a nonexistent file

16

FileOutputStream

- To construct a FileOutputStream, use the following constructors:

```
public FileOutputStream(String filename)
public FileOutputStream(File file)
public FileOutputStream(String filename, boolean append)
public FileOutputStream(File file, boolean append)
```

17

FileOutputStream

- If the file does not exist, a new file will be created
- If the file already exists, the first two constructors will delete (overwrite) the current contents in the file
- To retain the current content and append new data into the file, use the last two constructors by passing true to the append parameter

18

TestFileStream.java

```
try ( // Create an output stream to the file
      FileOutputStream output = new FileOutputStream("src/temp.dat");
  ) {
      // Output values to the file
      for (int i = 0; i <= 10; i++)
            output.write(i);
}
try ( // Create an input stream for the file
      FileInputStream input = new FileInputStream("src/temp.dat");
  ) {
      // Read values from the file
      int value;
      while ((value = input.read()) != -1)
            System.out.print(value + " ");
}
```

19

Activity

- Run TestFileStream.java and make sure it does something
- Try to view the temp.dat file in a text editor, what are the contents of the file?

20

Checking End of File

- Use input.read() to check for end of file
- input.read() == -1 indicates that it is the end of a file
- Imagine there was a file of binary prices that needed reading

21

ReadPriceFileStream.java

```
try (      // Create an input stream for the file
      FileInputStream input =
            new FileInputStream("src/pricelist.dat");
) {
    // Read values (only get int though!)
    double value;
    while ((value = input.read()) != -1)
          System.out.println(value);
} catch (Exception e) {
      System.out.println("An error occurred");
} finally {
      System.out.println("All processing done.");
}
```

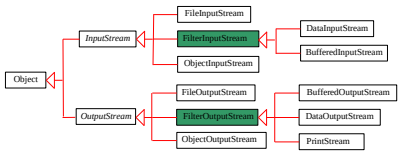
22

Question

- Given the current classes, is there any way to read/write binary files with anything other than bytes (technically ints)?

23

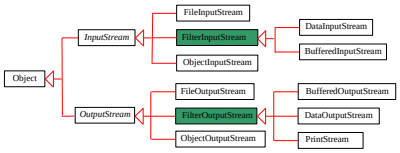
FilterInputStream/FilterOutputStream



- Filter streams are streams that filter bytes for some purpose
- Basic byte input stream provides a read method that can only be used for reading bytes
- If you want to read integers, doubles, or strings, you need a filter class to wrap the byte input stream

24

FilterInputStream/FilterOutputStream

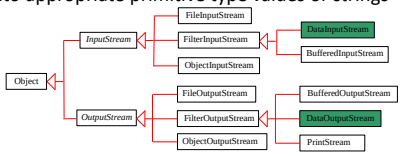


- Using a filter class enables you to read integers, doubles, and strings instead of bytes and characters
- `FilterInputStream` and `FilterOutputStream` are the base classes for filtering data
- When you need to process primitive numeric types, use `DataInputStream` and `DataOutputStream` to filter bytes

25

DataInputStream/DataOutputStream

`DataInputStream` reads bytes from the stream and converts them into appropriate primitive type values or strings

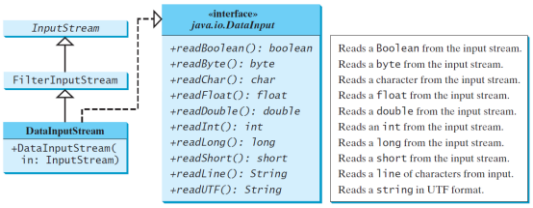


`DataOutputStream` converts primitive type values or strings into bytes and output the bytes to the stream

26

DataInputStream

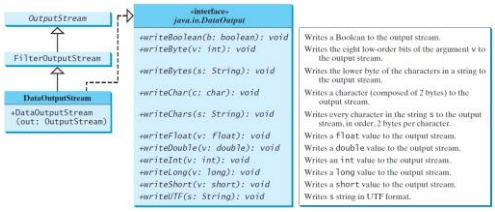
- `DataInputStream` extends `FilterInputStream` and implements the `DataInput` interface



27

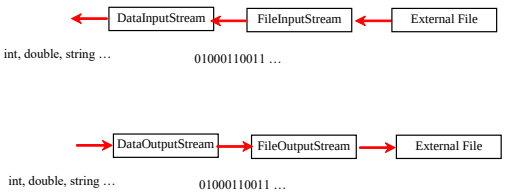
DataOutputStream

- `DataOutputStream` extends `FilterOutputStream` and implements the `DataOutput` interface



28

Pipeline Concept for Files



29

Using DataOutput/InputStream

- Imagine processing plain-text file grades
- Could take text file and convert it into binary file
- Once in binary format, could still process (very) similarly

30

Using DataOutput/InputStream

- Imagine converting a text file into a binary file to take advantage of binary format
- Read in text file using Scanner object, use DataOutputStream to output binary file

31

CreateBinaryPrices.java

```
try (
      Scanner input = new Scanner(new File("src/pricelist.txt"));
      DataOutputStream output = new DataOutputStream(
                  new FileOutputStream("src/pricelist.dat"));
) {
     while (input.hasNext()) {
        output.writeDouble(input.nextDouble());
     }
} catch (EOFException e) {
      System.out.println("Reached end of file");
} catch (Exception e) {
      e.printStackTrace();
}
```

32

Checking End of File

- **TIP:** If you keep reading data at the end of these basic streams, an `EOFException` will occur
  - Scanner could check `hasNext()` to prevent
- Often just use some form of an infinite loop and catch the `EOFException` to determine when you are done processing a `DataInputStream`

33

Using DataOutput/InputStream

- Imagine processing a binary file to make changes and store into another binary file
- Read in binary file using DataInputStream object, use DataOutputStream to output binary file

34

Process Prices

```
try ( // Create an input stream for file pricelist.dat
       DataInputStream input = new DataInputStream(
                    new FileInputStream("src/pricelist.dat"));
       DataOutputStream output = new DataOutputStream(
              new FileOutputStream("src/pricelist-new.dat"));
) {
       while (true) {  // Read from the file and add $10
              output.writeDouble(10 + input.readDouble());
       }
} catch (EOFException ex) {
       System.out.println("Reached end of file");
} catch (Exception ex) {
       System.out.println("Error");
}
```

35

Using DataOutput/InputStream

- How do you know if the program is working?
- Good idea to build class to read and display values from binary files
- Use DataInputStream and just print values to the console

36

ReadBinaryPrices.java

```
try ( // Create an input stream
      DataInputStream input = new DataInputStream(
                        new FileInputStream("src/pricelist.dat"));
) {
      while (true) {// Read student prices from the file
            System.out.println(input.readDouble());
      }
} catch (EOFException ex) {
      System.out.println("Reached end of file");
}
```

37

Characters and Strings in Binary I/O

- A Unicode character consists of two bytes
- writeChar(char c) method writes the Unicode of character c to the output
- writeChars(String s) method writes the Unicode for each character in the string s to the output.

38

Characters and Strings in Binary I/O

- Binary I/O uses UTF-8 for characters/strings
- UTF-8 is a coding scheme that allows systems to operate with both ASCII and Unicode efficiently
- Most operating systems use ASCII while Java uses Unicode (ASCII is a subset of Unicode)

39

Characters and Strings in Binary I/O

- Most applications only need ASCII character set, space is wasted when representing 8-bit ASCII char as 16-bit Unicode char
- UTF-8 is an alternative scheme that stores a character using 1, 2, or 3 bytes
- ASCII (less than 0x7FF) are coded in one byte
- Unicode less than 0x7FF are coded in two bytes
- Other Unicode values are coded in three bytes

40

DataInputStream/DataOutputStream

- Data streams are used as wrappers on existing streams to filter data in the original stream
- Created using the following constructors:

```
public DataInputStream(InputStream instream)
public DataOutputStream(OutputStream outstream)
```

Examples:

```
DataInputStream infile =
      new DataInputStream(new FileInputStream("in.dat"));
DataOutputStream outfile =
      new DataOutputStream(new FileOutputStream("out.dat"));
```

41

TestDataStream.java

```
try ( // Create output stream for temp.dat
      DataOutputStream output = new DataOutputStream(
        new FileOutputStream("src/ch17/temp.dat"));
) {
      // Write student test scores to the file
      output.writeUTF("John");
      output.writeDouble(85.5);
      output.writeUTF("Jim");
      output.writeDouble(185.5);
      output.writeUTF("George");
      output.writeDouble(105.25);
}
```

42

TestDataStream.java

```
try ( // Create input stream for temp.dat
      DataInputStream input = new DataInputStream(
            new FileInputStream("src/ch17/temp.dat"));
) {
      // Read student test scores from the file
      System.out.println(input.readUTF() + " " +
                                 input.readDouble());
      System.out.println(input.readUTF() + " " +
                                 input.readDouble());
      System.out.println(input.readUTF() + " " +
                                 input.readDouble());
}
```

43

Order and Format

- **CAUTION:** You have to read the data in the same order and same format it was stored
- For example, since names are written in UTF-8 using writeUTF(), must read names using readUTF()

44

Activity

- Open the prices.dat file with a text editor
- Using the prices.dat file, write a Java program that reads in price data and prints that information to the console
- The information in the binary data file contains the price of objects as a double, one to a line

45

Activity

- Modify your program so that it only outputs prices greater than $50 and less than $100
- Modify your program so that instead of printing to the console, that data is written to a binary file named pricesProcessed.dat

46

Activity – Part 1

- Open the employeeData.dat file with a text editor
- Using the employeeData.dat file, write a Java program that reads in employee data and prints that information to the console
- The information in the binary data file contains the employee's number designation ("Employee #") as UTF, their salary as a double, and their years of service as a double

47

Activity – Part 2/3

- Modify your program so that it only outputs people who have a salary greater than $20,000 and more than 2 years of service
- Modify your program so that instead of printing to the console, that data is written to a binary file named employeesProcessed.dat

48

Announcements

- Revel Exercises due
  - Chap 17
- Project 8 due Friday, start of lab
  - See Program Submission requirements in Folio

49